# Automating Continuous Household Tidying via Mobile Manipulation

Final-Project Proposal for *Robotics*

Wenbo Ma, Arif Onur Alp, Emir Pisirici
RWTH Aachen

Due: July 11, 2025

**Abstract**

We propose a *PyRoboSim*-based mobile robot that *continuously* tidies a domestic environment by fetching clutter (dirty dishes, laundry, small toys) from their spawning locations and depositing each item in its designated receptacle. A script-generated PDDL domain captures the pick–navigate–place loop, enabling a symbolic planner (Fast Downward) to produce high-level task sequences on demand. Motion is decomposed into three layers: (i) a global **RRT*** path computed with OMPL to connect semantic way-points across rooms; (ii) a real-time grid-based **A*** planner that refines each leg of the route; and (iii) a reactive **Bug2** controller to escape from unforeseen obstacles. Each area in the environment holds a probability score, which is dynamically updated from recent traversals, Objects are classified by a pretrained model, while their locations emerge online through the robot's continuous exploration. PyRoboSim supplies ground-truth object coordinates, so that no external perception module is required, allowing us to focus on planning, control, and life-cycle integration under ROS 2.

## 1 Research Question & Motivation

**Problem.** How can a low-cost mobile robot *continuously* locate, collect, and deliver daily clutter—dirty dishes from the living room and soiled laundry from bedrooms—to their appropriate household stations without human supervision?

**Relevance.** Domestic tidying consumes ≈1 h/day in the average European home. Automating the chore improves quality of life and forms a realistic benchmark for service-robot research.

**Course linkage.** Table 1 shows how each of the five lab exercises underpins the proposed system.

## 2 Proposed Solution

### 2.1 System Architecture

**World model.** A PyRoboSim YAML world with rooms, doorways, tables, laundry baskets, and a docking station, augmented by *temporal clutter generators* that spawn dishes/laundry via a Poisson process ($\lambda = 1/5$ hour).

**Symbolic layer.** PDDL domain with actions `navigate`, `pick`, `place`, `dropoff` defined in STRIPS form; plans produced by FastDownward via PyRoboSim's planner API.

**Motion layer.** • **Global**: RRT* between semantic way-points.

| Ex. | Technique | Role in Project |
|---|---|---|
| 1 | 2-D kinematics, joint chains | Base + 2-DOF arm forward/inverse transforms |
| 2 | Sampling motion planning (RRT*) | Global navigation between rooms |
| 3 | A*, Bug2 | Local obstacle avoidance and path repair |
| 4 | STRIPS / PDDL | Symbolic task sequencing (*collect*→*drop-off*) |
| 5 | PyRoboSim/ROS 2 integration | End-to-end execution and monitoring |

Table 1: Pedagogical roots of the project.

- **Local**: A* on a 0.05 m occupancy grid; Bug2 fallback whenever dynamic obstacles break heuristic guarantees.

**Execution monitor.** ROS 2 action executor.

**Continuity.** A lifecycle node that puts the robot in service every hour.

### 2.2 Algorithm–Tool Mapping

| Function | Algorithm / Library | Ex. |
|---|---|---|
| Forward kinematics | Custom Python matrices | 1 |
| Global navigation | `RRT` | 2 |
| Local avoidance | `Bug2/A*` | 3 |
| Task planning | `PDLL, STRIPS` | 4 |
| Simulation | PyRoboSim | 5 |

## 3 Probabilistic Map–Update Example

We illustrate the belief-map update mechanism with a concise two-frame scenario (Figures 2 and 3).

**Frame1 — Initial map.** Figure2 depicts the known world annotated with the robot's *a priori* probability distribution for encountering target objects. Cells shaded red are searched first, orange cells next, and green cells last.

**Frame2 — Experience-driven update.** After one day of operation the robot detects more than ten relevant objects inside a low-priority (green) region. The cell is therefore promoted to orange (Figure3). Analogously, any orange cell that accumulates 100+ detections is escalated to red. This threshold-based adaptation lets the system continuously realign its search strategy with real-world statistics.

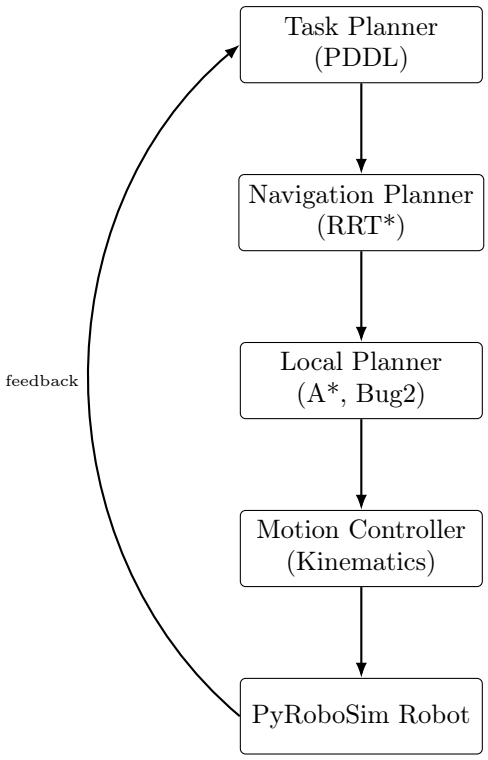


Figure 1: Layered architecture connecting symbolic planning to execution in PyRoboSim.

# 4 Success Criteria & Evaluation

| **Metric** | **Target** | **Measurement** |
|---|---|---|
| Task completion | $\geq$90% items delivered in a service round | Logged world-state queries |
| Planning time | Symbolic $<$ 10s; global path $<$ 5s | ROS 2 timestamps |
| Path optimality | Less then predefined max time | Compare to heuristic bound |
| Git hygiene | $\geq$90% unit-test coverage; green CI | GitLab Actions |

A run is **successful** if all four metrics meet their targets.

# 5 Project Plan & Management

## 5.1 Timeline

| **Week** | **Milestone** | **Deliverables** |
|---|---|---|
| 1 | World-building | YAML house, clutter script |
| 2 | Global planner | RRT* node, benchmark notebook |
| 3 | Local planner | A* grid & Bug2, collision tests |
| 4 | PDDL domain | STRIPS operators, unit plans |
| 5 | Integration | End-to-end demo video |
| 6 | Evaluation | Metric harness, stress tests |
| 7 | Report & polish | Final paper, code freeze |

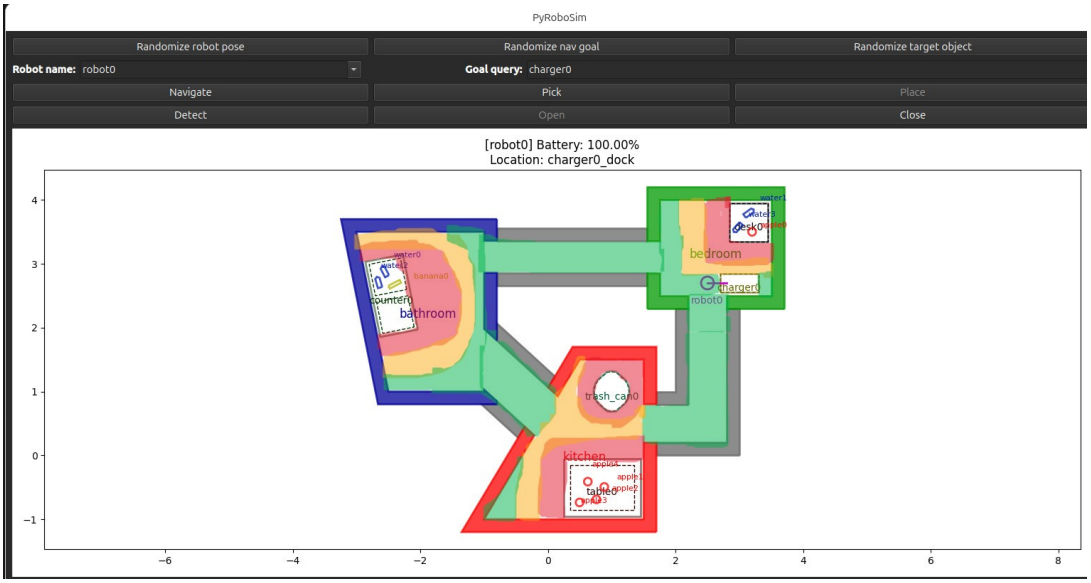


Figure 2: Initial probability map with red = high, orange = medium, green = low search priority.

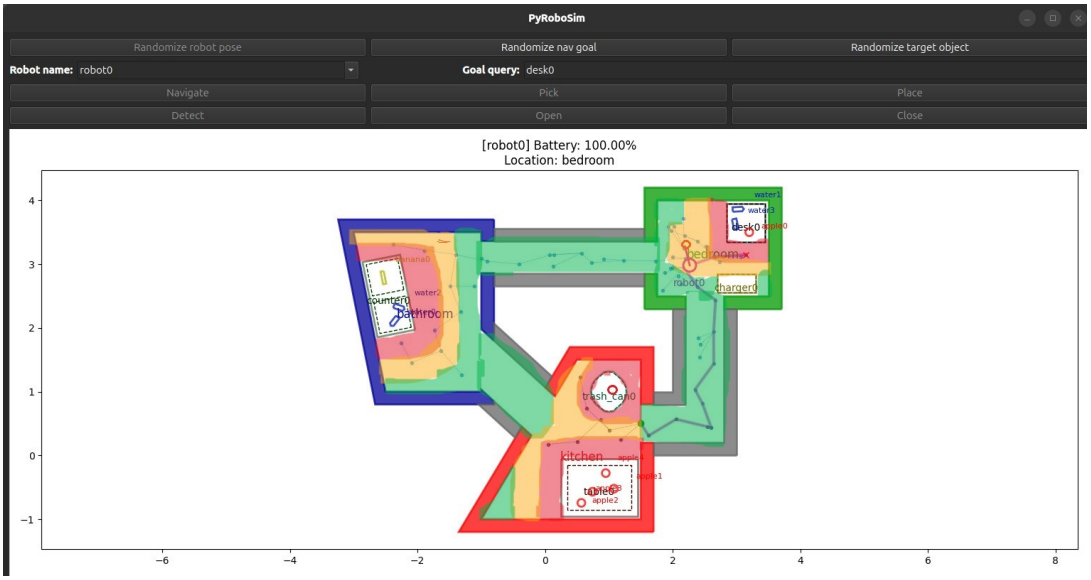


Figure 3: Map after daily update: a previously low-priority region has been promoted to orange after 10+ object detections. Regions exceeding 100 detections become red.

### 5.2 Tools & Practices

- **Version control:** GitLab, protected `main`, squash PRs.
- **CI:** GitLabActions—lint, pytest, headless simulation, coverage.
- **Issue tracking:** GitLab Projects kanban, label taxonomy.
- **Docs:** MkDocs auto-deployed to GitLab Pages.
- **Risk mitigation:** weekly 20-min review of codes.

## 6 References

## References


[1] S. M. LaValle, "Rapidly-Exploring Random Trees: A New Tool for Path Planning," 1998.

[2] R. E. Fikes and N. J. Nilsson, "STRIPS: A New Approach to the Application of Theorem Proving to Problem Solving," *Artificial Intelligence*, 1971.

[3] P. E. Hart, N. J. Nilsson, and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum-Cost Paths," *IEEE Transactions on Systems Science and Cybernetics*, 1968.

[4] H. Choset, "Bug Algorithms," CMU Lecture Notes, 2010.

[5] S. Castro, *pyrobosim*: ROS 2 Enabled 2-D Mobile-Robot Simulator, GitHub, 2025.

[6] *PyRoboSim Documentation*, v4.1.0, 2025.

[7] L. Kavraki *et al.*, "Probabilistic Roadmaps for Path Planning," *IEEE Transactions on Robotics and Automation*, 1996.

[8] Robohub, "Building a Python Toolbox for Robot Behavior: pyrobosim," 2022.